

SYSTEM DESIGN DOCUMENT — E-commerce Marketplace

Thông tin tài liệu	Giá trị
Mã tài liệu	SDD-MKTPLACE-CORE-v1.0
Phiên bản	1.0.0
Trạng thái	☒ Approved
Ngày tạo	06/06/2026
Cập nhật lần cuối	06/06/2026
Tác giả	[Họ tên]
Người phê duyệt	[Họ tên]
Dự án / Hệ thống	E-commerce Marketplace (multi-merchant)
Mức độ bảo mật	☒ Internal

Lịch sử thay đổi

Phiên bản	Ngày	Tác giả	Mô tả thay đổi	Người duyệt
1.0.0	06/06/2026	[Tác giả]	Tạo mới tài liệu	[Duyệt]

Mô hình 3 tài liệu: SDD này (SAD — tầng kiến trúc) tham chiếu *lên* Standard (chuẩn org-wide) và *xuống* các **Tech Spec** (thiết kế chi tiết tầng 3). Đặc tả API → OpenAPI; schema cột → Tech Spec/migration; giá trị & secret → config/Vault. Bảng tương ứng SAD↔Tech Spec ở Phụ lục A.

1. TỔNG QUAN HỆ THỐNG

1.1 Mục tiêu hệ thống

#	Mục tiêu	Mô tả	KPI
M1	Vận hành sàn multi-merchant	Buyer mua từ nhiều Merchant trong một phiên; hệ tự tách đơn theo Merchant	Hỗ trợ ≥ 10.000 Merchant; tách đơn chính xác 100%
M2	Thanh toán an toàn qua Escrow	Giữ tiền đến khi đơn hoàn tất, bảo vệ Buyer & Merchant	0 sự cố mất/lệch tiền; đối soát khớp 100%
M3	Đối soát & chi trả tự động	Tính phí sàn/hoa hồng, payout cho Merchant	Payout đúng hạn ≥ 99%; chứng từ bất biến
M4	Kiểm duyệt & nguồn sự thật sản phẩm	Catalog là source of truth cho hiển thị/tìm kiếm	Thời gian duyệt < 24h; search P95 < 200ms

#	Mục tiêu	Mô tả	KPI
M5	Chịu tải cao điểm	Flash sale, sự kiện	3.000 RPS sustained, checkout P99 < 800ms

1.2 Phạm vi hệ thống

1.2.1 Trong phạm vi

- Quản lý tài khoản người dùng, xác thực danh tính và phân quyền theo vai trò (Buyer, Merchant, Admin).
- Quản lý danh mục sản phẩm, thương hiệu; kiểm duyệt nội dung trước khi hiển thị và hỗ trợ tìm kiếm toàn văn.
- Theo dõi tồn kho, giữ chỗ hàng hóa khi đặt đơn (reservation) và trừ kho vĩnh viễn khi đơn hoàn tất.
- Điều phối quy trình đặt hàng: tự động tách đơn theo từng nhà bán hàng, tính giá và áp dụng chương trình khuyến mãi.
- Quản lý toàn bộ vòng đời đơn hàng thông qua máy trạng thái (state machine) từ lúc tạo đến khi hoàn thành hoặc hủy.
- Xử lý giao dịch thanh toán qua cổng bên thứ ba, giữ tiền trung gian cho đến khi giao hàng thành công (escrow), đối soát tự động và chi trả cho nhà bán hàng.

1.2.2 Ngoài phạm vi

- **Vận chuyển vật lý** – dùng đơn vị vận chuyển ngoài (Courier) – chỉ tích hợp, không kiểm soát.
- **Kế toán/ERP doanh nghiệp** – nhận dữ liệu đối soát qua file/API, ngoài phạm vi.
- **BI/Analytics, CSKH/ticketing** – hệ riêng.
- **Rating & Dispute** – chưa triển khai ở v1.0 (xem ADR-0012 cho lộ trình bổ sung).

1.2.3 Sơ đồ ngữ cảnh (Context Diagram – C4 Level 1)

Ở L1: chỉ "nối ai, để làm gì" — không protocol/công nghệ.

```
graph TD
    Buyer[Khách hàng - Buyer]
    Merchant[Nhà bán hàng - Merchant]
    Admin[Platform Admin]
    System((HỆ THỐNG MARKETPLACE))
    PG[Cổng thanh toán]
    Bank[Ngân hàng Merchant]
    Courier[Đơn vị vận chuyển]
    Notif[Email/SMS provider]

    Buyer -->|mua hàng| System
    Merchant -->|bán, xử lý đơn| System
    Admin -->|kiểm duyệt, xử lý| System
    System -->|thanh toán & payout| PG
    System -->|chi trả| Bank
    System -->|tạo vận đơn| Courier
    System -->|thông báo| Notif
```

Hệ thống ngoài	Loại tương tác	Mô tả
Cổng thanh toán	API + Webhook	Thu tiền Buyer, callback kết quả
Ngân hàng Merchant	API	Payout sau đối soát
Đơn vị vận chuyển	API	Tạo vận đơn, theo dõi giao hàng
Email/SMS provider	API	Thông báo trạng thái đơn

1.3 Các bên liên quan (Stakeholders)

Bên liên quan	Vai trò	Loại	Kỳ vọng
Buyer	End User	External	Mua nhanh, tiền an toàn (escrow)
Merchant	End User	External	Nhận đơn kịp thời, payout đúng
Platform Admin	Operator	Internal	Kiểm duyệt, xử lý tranh chấp
Finance team	Interested	Internal	Đối soát khớp, chứng từ bất biến
Security team	Interested	Internal	Cô lập tenant, audit đầy đủ
SRE/Ops	Interested	Internal	Để vận hành, giám sát

1.4 Giả định & Ràng buộc

1.4.1 Giả định

- Cổng thanh toán hỗ trợ giữ tiền/escrow hoặc cho phép mô phỏng escrow phía sàn.
- Cao điểm ≤ 3.000 RPS giai đoạn đầu.
- Mỗi Buyer có thể mua từ nhiều Merchant trong một giỏ → bắt buộc tách đơn.

1.4.2 Ràng buộc

Loại	Ràng buộc	Tác động thiết kế
Kỹ thuật	PostgreSQL chuẩn org; Kafka làm event bus	DB-per-context, ORM, schema design
Pháp lý	NĐ 13/2023 (PII); chứng từ tài chính phải lưu bất biến	Object Lock/WORM cho S3 result bucket; data residency
Tài chính	Tiền trong escrow là tiền thật của khách	Idempotency + audit bắt buộc cho mọi thao tác tiền
Tổ chức	Multi-tenant (nhiều Merchant)	Cô lập dữ liệu theo tenant xuyên suốt authz

⚠ **Open item (TBD):** Rule phân quyền chi tiết cho S3 result bucket (chứng từ đối soát) — nguyên tắc đã chốt (*prevent overwrite/override*, WORM), nhưng IAM policy cụ thể đang chờ; theo dõi bằng ADR riêng. Xem mục 6.4.3 & 7.3.

2. KIẾN TRÚC TỔNG THỂ

2.1 Kiểu kiến trúc

Kiểu	Lý do	Trade-off
Microservices theo Bounded Context (DDD) + Orchestration (Checkout) + Event-Driven	Mỗi context scale & deploy độc lập; Checkout làm điều phối cho luồng phức tạp; event cho liên kết lỏng lẻo	Phức tạp phân tán: cần saga/compensation, distributed tracing, eventual consistency

2.1.1 Nguyên tắc thiết kế kiến trúc

- **Database per Context** – không chia sẻ DB, không FK xuyên context.
- **API-First** – contract rõ ràng (REST ngoài, gRPC nội bộ giữa context).
- **Orchestration cho luồng tiền** – Checkout điều phối đồng bộ; sai bước → compensation (saga).
- **Idempotency cho mọi thao tác tiền** – escrow, payout, webhook.
- **Tenant isolation** – mọi truy vấn gắn `merchant_id/owner`; Merchant chỉ thấy dữ liệu của mình.
- **Chứng từ tài chính bất biến** – ghi một lần, không sửa/xóa (WORM).
- **Secrets ở Vault**, không trong tài liệu/code.

2.2 Sơ đồ kiến trúc cấp cao (C4 Level 2 – Container Diagram)

Có công nghệ + external systems ở rìa + ranh giới tin cậy.

```
graph TD
  subgraph Internet [INTERNET - untrusted]
    Client[Client Apps: Buyer/Merchant/Admin]
  end
  Gateway[API Gateway / BFF - Kong]

  subgraph VPC [VPC noi bo - service-to-service mTLS]
    Identity[Identity Svc - Node.js+Keycloak]
    Catalog[Catalog Svc - Spring Boot + Elasticsearch]
    Inventory[Inventory Svc - Go]
    Checkout[Checkout Svc - Spring Boot - ORCHESTRATOR]
    Order[Order Svc / OMS - Spring Boot]
    Payment[Payment Svc - Node.js - Escrow/Settlement]
    Notif[Notification Svc - Python]
    Bus{Kafka Event Bus}
  end

  Client -->|HTTPS/REST| Gateway
  Gateway -->|auth/route| Identity
  Gateway --> Catalog
  Gateway --> Checkout
  Gateway --> Order
  Gateway --> Payment
```

```
Checkout ==>|gRPC GetPrice| Catalog
Checkout ==>|gRPC ReserveStock| Inventory
Checkout ==>|gRPC CreatePendingOrder| Order
Checkout ==>|gRPC InitEscrow| Payment

Catalog -.->|ProductCreated| Bus
Payment -.->|PaymentReceived| Bus
Order -.->|OrderCompleted| Bus
Bus -.-> Order
Bus -.-> Inventory
Bus -.-> Payment
Bus -.-> Notif

Payment ==>|HTTPS+HMAC outbound| PG[Cong thanh toan - external]
PG -.->|webhook inbound - verify chu ky| Payment
Payment ==>|payout| Bank[Ngan hang - external]
Notif ==>|HTTPS| Provider[Email/SMS - external]
```

Container	Công nghệ	Vai trò	Giao tiếp
API Gateway/BFF	Kong	Routing, authn, rate limit, TLS	HTTPS → services
Identity Svc	Node.js + Keycloak	Authn, RBAC 3 vai trò	OIDC/JWT
Catalog Svc	Spring Boot + Elasticsearch	Sản phẩm, kiểm duyệt, search	REST + gRPC + Kafka
Inventory Svc	Go	Tồn kho, reserve, trừ kho	gRPC + Kafka
Checkout Svc	Spring Boot	Orchestrator: tách đơn, điều phối	gRPC → các context
Order Svc (OMS)	Spring Boot	State machine đơn	REST + Kafka
Payment Svc	Node.js	Escrow, đối soát, payout	gRPC + Kafka + HTTPS→PG/Bank
Notification Svc	Python	Email/SMS	Kafka + HTTPS
Event Bus	Kafka	Sự kiện bất đồng bộ	Kafka protocol

2.3 Mô hình triển khai

2.3.1 Môi trường triển khai

Môi trường	Hạ tầng	Đặc điểm
Development	Docker Compose	Dữ liệu giả
Staging	K8s namespace	Mirror prod, dữ liệu ẩn danh

Môi trường	Hạ tầng	Đặc điểm
Production	K8s multi-AZ	HA, auto-scaling, full monitoring

2.3.2 Sơ đồ triển khai Production

Topology: CDN → LB → API Gateway (multi-AZ) → service pods (HPA) → PostgreSQL (primary + read replica per context), Kafka cluster, Redis, S3 (ảnh + result bucket WORM). Payment Svc đặt trong subnet hạn chế egress (chỉ tới PG/Bank).

```
graph TB
    classDef ext fill:#fde,stroke:#a33,stroke-width:1px;
    classDef data fill:#eef,stroke:#36c,stroke-width:1px;
    classDef restricted fill:#ffe9e0,stroke:#e8590c,stroke-width:2px,stroke-dasharray:5 5;

    Client[Client Apps  
Buyer / Merchant / Admin]
    CDN[CDN  
static + product images]

    subgraph Edge[Edge layer - Multi-AZ]
        LB[Load Balancer  
multi-AZ]
        GW[API Gateway pods  
Kong - HPA]
    end

    subgraph K8s[K8s Cluster - Multi-AZ, auto-scaling]
        subgraph App[App zone - service pods HPA]
            Identity[Identity]
            Catalog[Catalog]
            Inventory[Inventory]
            Checkout[Checkout]
            Order[Order]
            Notif[Notification]
        end
        subgraph Restricted[Restricted egress subnet]
            Payment[Payment Svc  
egress chỉ tới PG / Bank]
        end
    end

    subgraph Data[Data layer]
        PG[(PostgreSQL per-context  
primary + read replica)]
        ES[(Elasticsearch  
search index)]
        Kafka[Kafka cluster]
        Redis[(Redis  
cache / session)]
        S3img[S3 - product images]
```

```

    S3worm[S3 result bucket
    WORM / Object Lock]
  end

  subgraph Ext[Hệ thống ngoài]
    PGext[Cổng thanh toán]
    Bank[Ngân hàng Merchant]
    Courier[Đơn vị vận chuyển]
    Provider[Email / SMS]
  end

  Client -->|HTTPS| CDN
  Client -->|HTTPS| LB
  CDN -->|cache miss| S3img
  LB --> GW
  GW --> Identity & Catalog & Inventory & Checkout & Order & Payment

  Identity --> PG
  Catalog --> PG
  Catalog --> ES
  Inventory --> PG
  Order --> PG
  Payment --> PG
  Checkout --> Redis
  Catalog --> Kafka
  Order --> Kafka
  Payment --> Kafka
  Notif --> Kafka
  Catalog --> S3img
  Payment --> S3worm

  Payment ==>|HMAC, egress allow| PGext
  PGext -->|webhook inbound| LB
  Payment ==>|payout| Bank
  Notif --> Provider
  Order --> Courier

  class PGext,Bank,Courier,Provider ext;
  class PG,ES,Kafka,Redis,S3img,S3worm data;
  class Payment restricted;

```

2.3.3 Chiến lược triển khai

Rolling update mặc định; Canary cho Checkout/Payment (rủi ro cao); Feature flag cho khuyến mãi; Blue/Green khi đổi schema breaking.

3. CÁC THÀNH PHẦN HỆ THỐNG

3.1 Tổng quan các thành phần

#	Component	Loại	Công nghệ	Trách nhiệm chính
1	API Gateway/BFF	Infra	Kong	Điều phối lưu lượng từ client tới các service nội bộ; xác thực JWT đầu vào; áp rate limit & WAF bảo vệ tầng biên; gắn tenant scope cho mọi request; TLS termination.
2	Identity	Microservice	Node.js+Keycloak	Quản lý vòng đời tài khoản người dùng (Buyer, Merchant, Admin); cấp & xác thực JWT (RS256) qua OIDC; phân quyền RBAC ba vai trò; hỗ trợ MFA cho Admin & Merchant rút tiền; quản lý phiên SSO.
3	Catalog	Microservice	Spring Boot+ES	Quản lý danh mục sản phẩm, biến thể, SKU & thương hiệu (source of truth); kiểm duyệt nội dung trước khi hiển thị; lập chỉ mục & tìm kiếm toàn văn qua Elasticsearch; phát sự kiện ProductCreated để đồng bộ Inventory.
4	Inventory	Microservice	Go	Theo dõi số lượng tồn kho theo SKU; giữ chỗ hàng hoá khi đặt đơn (reservation); trừ kho vĩnh viễn khi đơn hoàn tất (consume sự kiện OrderCompleted); giải phóng reservation khi saga compensation.
5	Checkout	Microservice	Spring Boot	Điều phối (orchestrator) toàn bộ quy trình đặt hàng: tổng hợp giá từ Catalog, reserve kho từ Inventory, tự động tách đơn theo từng Merchant, tạo pending order, khởi tạo escrow; thực thi saga compensation khi có bước lỗi.
6	Order (OMS)	Microservice	Spring Boot	Quản lý toàn bộ vòng đời đơn hàng qua máy trạng thái (Pending → Paid → To Ship → Shipped → Completed / Cancelled); phát sự kiện OrderCompleted kích hoạt trừ kho & settlement; lưu snapshot giá tại thời điểm đặt.
7	Payment	Microservice	Node.js	Xử lý giao dịch qua cổng thanh toán bên thứ ba; giữ tiền trung gian (escrow) đến khi giao hàng thành công; tính phí sàn & hoa hồng; thực hiện payout cho Merchant; sinh chứng từ đối soát bất biến (WORM/S3); xác minh webhook (chữ ký + idempotency).
8	Notification	Microservice	Python	Lắng nghe sự kiện từ Kafka (PaymentReceived, OrderCompleted...); gửi thông báo đa kênh (email, SMS) qua provider bên thứ ba; hỗ trợ fallback kênh thay thế khi gửi thất bại.

3.2 Mô tả chi tiết từng thành phần

3.2.1 Checkout Service (Orchestrator)

Thuộc tính	Giá trị
Loại / Công nghệ / Owner	Microservice / Spring Boot / Checkout team
Ranh giới tin cậy	Nội bộ; điều phối đồng bộ tới Catalog/Inventory/Order/Payment
Trạng thái	Gần như stateless; phiên checkout tạm ở Redis
Trách nhiệm	Tổng hợp giá (Catalog), reserve kho (Inventory), tách đơn theo Merchant, tạo pending order (Order), init escrow (Payment); compensation khi một bước lỗi.
Thiết kế chi tiết	TechSpec-Marketplace-Checkout.md

3.2.2 Payment Service (Escrow & Settlement)

Thuộc tính	Giá trị
Loại / Công nghệ / Owner	Microservice / Node.js / Payment team
Ranh giới tin cậy	Outbound → cổng thanh toán & ngân hàng; inbound webhook; ghi chứng từ bất biến lên S3 result bucket
Trách nhiệm	giao dịch cổng ngoài, escrow (giữ tiền), tính phí/hoa hồng, payout; sinh chứng từ đối soát (WORM).
Thiết kế chi tiết	TechSpec-Marketplace-Payment.md

4. LUỒNG DỮ LIỆU

4.1 Luồng chính (Happy Path)

4.1.1 Checkout & tách đơn (Orchestration)

```
sequenceDiagram
    actor Buyer
    participant Checkout
    participant Catalog
    participant Inventory
    participant Order
    participant Payment
    Buyer->>Checkout: 1. Submit checkout (giỏ nhiều Merchant)
    Checkout->>Catalog: 2. Get price
    Catalog-->>Checkout: 3. Price
    Checkout->>Inventory: 4. Reserve stock
    alt Hết hàng
```

```

    Inventory-->>Checkout: 5a. Fail
    Checkout-->>Buyer: 5b. Error (không tạo đơn)
else Còn hàng
    Inventory-->>Checkout: 6. Reserved
    Checkout-->>Order: 7. Create pending orders (TÁCH theo Merchant)
    Order-->>Checkout: 8. Order IDs
    Checkout-->>Payment: 9. Init escrow (tổng giỏ)
    Payment-->>Checkout: 10. Payment URL
    Checkout-->>Buyer: 11. Redirect cổng thanh toán
end

```

4.1.2 Thanh toán → cập nhật đơn

Cổng thanh toán → Payment webhook (verify chữ ký) → Payment giữ tiền (escrow) → phát **PaymentReceived** → Order chuyển "To Ship" → Notification báo Merchant.

4.1.3 Hoàn tất & đối soát (Settlement)

Buyer xác nhận nhận hàng → Order "Completed" → phát **OrderCompleted** → Inventory trừ kho vĩnh viễn; Payment tính hoa hồng → release escrow → payout về ngân hàng Merchant → sinh chứng từ đối soát (ghi **bất biến** lên S3 result bucket).

4.2 Data Flow Diagram (mức loại dữ liệu)

4.2.1 Luồng Nhà bán hàng & Quản trị (Merchant & Admin Flow)

```

flowchart TD
    classDef entity fill:#f9f6f0,stroke:#333,stroke-width:2px;
    classDef process fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    classDef datastore fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    classDef note fill:#fff9c4,stroke:#fbc02d,stroke-width:1px,stroke-dasharray: 5
    5;

    %% Entities
    Merchant["Nhà bán hàng / Merchant"]:::entity
    Admin["Platform Admin"]:::entity
    Bank["Merchant Bank"]:::entity

    %% Processes
    P_Catalog(("Catalog Context")):::process
    P_Inventory(("Inventory Context")):::process
    P_Order(("Order Context")):::process
    P_Payment(("Payment Context")):::process

    %% Datastores
    D_Catalog[("Catalog DB")]:::datastore
    D_Inventory[("Inventory DB")]:::datastore
    D_Order[("Order DB")]:::datastore
    D_Payment[("Payment DB & S3 result buckets")]:::datastore

    %% Note

```

```

N2>"Bucket lưu trữ chứng từ bắt buộc phải cấu hình
prevent overwrite hoặc override để bảo vệ dữ liệu"]:::note
D_Payment -- N2

%% Data Flows
Merchant -->|"1. Khai báo thuộc tính SP & Biển thể"| P_Catalog
Admin -->|"2. Dữ liệu phê duyệt hiển thị"| P_Catalog
P_Catalog -->|"3. Lưu trữ cấu trúc Cây danh mục & SP"| D_Catalog
P_Catalog -->|"4. Luồng sự kiện ProductCreated"| P_Inventory
P_Inventory -->|"5. Khởi tạo mã SKU rỗng"| D_Inventory

Merchant -->|"6. Nhập/Cập nhật số lượng khả dụng"| P_Inventory
P_Inventory -->|"7. Ghi đề biến động tồn kho"| D_Inventory

P_Order -->|"8. Thông báo có đơn hàng mới (Paid)"| Merchant
Merchant -->|"9. Gửi dữ liệu xác nhận đã giao ĐVVC"| P_Order
P_Order -->|"10. Cập nhật trạng thái Shipped"| D_Order

P_Payment -->|"11. Tính toán chiết khấu & hoa hồng sần"| D_Payment
P_Payment -->|"12. Giải phóng quỹ Escrow (Release)"| D_Payment
P_Payment -->|"13. Xuất lệnh chuyển tiền Payout"| Bank

```

4.2.2 Luồng Khách hàng (Buyer Flow)

```

flowchart TD
classDef entity fill:#f9f6f0,stroke:#333,stroke-width:2px;
classDef process fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
classDef datastore fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
classDef note fill:#fff9c4,stroke:#fbc02d,stroke-width:1px,stroke-dasharray: 5
5;

%% Entities
Buyer[Khách hàng / Buyer]:::entity
Gateway[Payment Gateway]:::entity

%% Processes
P_Checkout((Checkout Context)):::process
P_Catalog((Catalog Context)):::process
P_Inventory((Inventory Context)):::process
P_Order((Order Context)):::process
P_Payment((Payment Context)):::process

%% Datastores
D_Inventory[(Inventory DB)]:::datastore
D_Order[(Order DB)]:::datastore
D_Payment[(Payment DB)]:::datastore

%% Note
N1>Cơ chế Retry khi lỗi đồng bộ mạng: TBD]:::note
P_Checkout -- N1

```

```
%% Data Flows
Buyer -->|1. Gửi thông tin Giỏ hàng & Checkout| P_Checkout
P_Checkout -->|2. Yêu cầu giá & Metadata| P_Catalog
P_Checkout -->|3. Yêu cầu giữ tồn kho| P_Inventory
P_Inventory -->|4. Cập nhật trạng thái khóa kho| D_Inventory

P_Checkout -->|5. Yêu cầu tạo đơn| P_Order
P_Order -->|6. Ghi dữ liệu đơn Pending| D_Order

P_Checkout -->|7. Yêu cầu tạo giao dịch Escrow| P_Payment
P_Payment -->|8. Trả về URL/Token Thanh toán| Buyer

Gateway -->|9. Webhook: Dữ liệu thanh toán thành công| P_Payment
P_Payment -->|10. Ghi nhận số dư Escrow| D_Payment
P_Payment -->|11. Luồng sự kiện PaymentReceived| P_Order
P_Order -->|12. Cập nhật trạng thái Paid| D_Order

Buyer -->|13. Xác nhận đã nhận hàng| P_Order
P_Order -->|14. Cập nhật trạng thái Completed| D_Order
P_Order -->|15. Luồng sự kiện OrderCompleted| P_Inventory
P_Inventory -->|16. Trừ số lượng kho vĩnh viễn| D_Inventory
P_Order -->|17. Lệnh Trigger Settlement| P_Payment
```

4.3 Luồng bất đồng bộ (Event-Driven)

Event	Publisher	Subscriber(s)	Schema tóm tắt	Retry/DLQ
ProductCreated	Catalog	Inventory	{productId, skus[]}	3 retries, catalog-dlq
PaymentReceived	Payment	Order, Notification	{orderId, txnId, amount}	5 retries, pay-dlq
OrderCompleted	Order	Inventory, Payment	{orderId, items[]}	5 retries, order-dlq

5. GIAO DIỆN HỆ THỐNG

Optional

```
graph TB
    classDef pub fill:#fde,stroke:#a33,stroke-width:2px;
    classDef vpc fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    classDef out fill:#fff3e0,stroke:#f57c00,stroke-width:2px;
    classDef inb fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;

    Client[Client Apps]:::pub

    subgraph B1[Ranh giới 1 - Public edge: authn + rate limit + tenant scope + WAF]
        GW[API Gateway]:::pub
    end

    subgraph B2[Ranh giới 2 - Inter-context: mTLS + service account]
```

```

    G1[Catalog.GetPrice]:::vpc
    G2[Inventory.ReserveStock]:::vpc
    G3[Order.CreatePendingOrder]:::vpc
    G4[Payment.InitEscrow]:::vpc
  end
  subgraph B3[Ranh gioi 3 - Outbound: HMAC + egress han che + secret Vault]
    O1[to Cong thanh toan]:::out
    O2[to Ngan hang - payout]:::out
  end
  subgraph B4[Ranh gioi 4 - Inbound webhook: verify chu ky + allowlist IP + idempotency]
    W1[POST /v1/payments/webhook]:::inb
  end

  Client -->|HTTPS REST + JWT| GW
  GW -->|route| G1 & G2 & G3 & G4
  G4 --> O1
  O1 --> W1
  G4 --> O2

```

5.1 Internal APIs

5.1.1 Quy ước chung

Quy ước	Áp dụng
Base URL	https://api.marketplace.com/v{version}
Auth	Bearer JWT (RS256), service-to-service mTLS
Internal RPC	gRPC giữa các context
Versioning	URL /v1/, hỗ trợ N-1
Error format	<code>{error:{code, message, details[]}}</code>
ID	UUID v4 · Time ISO 8601 UTC

5.1.2 Phân loại interface theo ranh giới tin cậy

Nhóm	Ranh giới	Cơ chế bảo vệ
Public (qua Gateway)	Internet→edge	Authn, rate limit, validation, tenant scope
Inter-context (gRPC)	VPC nội bộ	mTLS + service account
Outbound (PG/Bank)	Nội bộ→ngoài	HMAC, timeout, retry, secret ở Vault, egress hạn chế
Inbound webhook	Ngoài→nội bộ	Verify chữ ký, allowlist IP, idempotency

5.1.3 Danh sách API quan trọng & ví dụ

#	Loại	Interface	Auth	Mô tả
---	------	-----------	------	-------

#	Loại	Interface	Auth	Mô tả
1	gRPC	Catalog.GetPrice	mTLS	Lấy giá cho checkout
2	gRPC	Inventory.ReserveStock	mTLS	Giữ chỗ tồn kho
3	gRPC	Order.CreatePendingOrder	mTLS	Tạo đơn (tách Merchant)
4	gRPC	Payment.InitEscrow	mTLS	Khởi tạo giữ tiền
5	REST	POST /v1/payments/webhook	Chữ ký	Callback cổng thanh toán

Đặc tả đầy đủ → OpenAPI/proto mỗi service; chi tiết nghiệp vụ → Tech Spec tương ứng.

5.2 External Integration Points

Hệ thống	Loại	Ranh giới	Dữ liệu qua biên	Resilience	Compliance
Cổng thanh toán	Payment	Outbound + webhook	Số tiền, orderRef (không PAN)	Timeout 30s, fallback reconcile	Subprocessor, DPA
Ngân hàng Merchant	Payout	Outbound	STK (nhạy cảm), số tiền	Retry + đối soát thủ công	Hợp đồng ngân hàng
Đơn vị vận chuyển	Logistics	Outbound	Địa chỉ (PII)	Retry → queue	Subprocessor, DPA
Email/SMS	Notification	Outbound	SĐT/email (PII)	Fallback kênh khác	Subprocessor

6. KIẾN TRÚC DỮ LIỆU

6.1 Tổng quan kiến trúc dữ liệu

```
graph TB
  classDef owner fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
  classDef ref fill:#fff,stroke:#999,stroke-dasharray:4 3;
  classDef sensitive fill:#fde,stroke:#a33,stroke-width:2px;

  subgraph Identity[Identity Context]
    U[User, Role - PII]::sensitive
  end
  subgraph Catalog[Catalog Context - SOURCE OF TRUTH san pham]
    P[Product, Variant, SKU, Brand]::owner
  end
  subgraph Inventory[Inventory Context]
    Stock[Stock, Reservation]::owner
  end
  subgraph Order[Order Context]
    O[Order, OrderItem - PII dia chi]::sensitive
  end
  subgraph Payment[Payment Context]
    Pay[Payment, EscrowHold]::owner
```

```
Settle[Settlement, Payout - STK nhảy cam]:::sensitive
Doc[(Settlement Doc - S3 WORM bat bien)]:::sensitive
end

%% Reference logic xuyên context - KHONG FK vat ly
Order -->|ref user_id| U
Order -->|ref product_id, snapshot gia| P
Inventory -->|ref sku| P
Pay -->|ref order_id| O
Settle -->|ref merchant_id| U
Settle --> Doc
```

Context	Database	Loại	Lý do
Identity	identity_db	PostgreSQL	ACID, audit
Catalog	catalog_db + ES	PostgreSQL + Elasticsearch	Relational + search
Inventory	inventory_db	PostgreSQL	Consistency tồn kho
Order	order_db	PostgreSQL	ACID, state machine
Payment	payment_db + S3	PostgreSQL + S3 (WORM)	ACID + chứng từ bất biến
(chung)	Redis · S3	Cache/session · Object	Phiên checkout · ảnh sản phẩm

6.2 Entity Relationship Diagram (ERD)

Mỗi context có ERD riêng, không FK vật lý xuyên context. Tham chiếu xuyên context (vd **order_id** trong payment_db, **product_id** trong order_db) là *reference logic*. (ERD từng context → repo diagram + Tech Spec.)

6.3 Định nghĩa Schema chính

Tổ chức theo context. Schema cột chi tiết → Tech Spec tương ứng:

- Payment (payments, escrow_holds, settlements) → **TechSpec-Marketplace-Payment.md**
- Checkout/Order → **TechSpec-Marketplace-Checkout.md**

6.4 Chiến lược quản lý dữ liệu

6.4.1 Migration

Flyway, backward-compatible ≥ 1 deploy cycle.

6.4.2 Backup

DB chính daily full + hourly incremental (Payment tier-1: RPO < 5 phút).

6.4.3 Data Lifecycle & phân loại

Loại dữ liệu	Retention	Khi hết hạn	Cơ sở
--------------	-----------	-------------	-------

Loại dữ liệu	Retention	Khi hết hạn	Cơ sở
PII (hồ sơ, địa chỉ)	TK active + 30 ngày	Anonymize	NĐ 13/2023
Dữ liệu giao dịch/đơn	10 năm	Archive → purge	Luật kế toán
Chứng từ đối soát (S3)	10 năm	Immutable (WORM), không ghi đè/xóa	Compliance tài chính
Audit log	5 năm	Immutable (Object Lock)	Compliance

⚠ **TBD:** IAM policy chi tiết cho S3 result bucket (chứng từ đối soát) đang chờ chốt — nguyên tắc: write-once, deny overwrite/delete kể cả với owner; xem mục 7.3.

7. KIẾN TRÚC BẢO MẬT

Hướng theo ZTA

```
graph TB
    classDef cp fill:#e1f5fe,stroke:#0288d1,stroke-width:2px;
    classDef pep fill:#fff3e0,stroke:#f57c00,stroke-width:2px;
    classDef wl fill:#e8f5e9,stroke:#388e3c,stroke-width:2px;
    classDef user fill:#f9f6f0,stroke:#333,stroke-width:2px;

    User[User: Buyer/Merchant/Admin]:::user

    subgraph CP[CONTROL PLANE - nơi RA QUYET DINH]
        IdP[Identity Provider - OIDC/JWT]
    end
    danh tinh NGUOI DUNG]:::cp
    SPIRE[SPIFFE/SPIRE Server]
    cap SVID - danh tinh WORKLOAD]:::cp
    PDP[Policy Decision Point]
    PE + PA - danh gia policy authz]:::cp
    CA[Internal CA / cert rotation]:::cp
    end

    subgraph DP[DATA PLANE - nơi THUC THI moi request]
        GW[API Gateway - PEP]
    end
    verify JWT + tenant scope]:::pep
    subgraph Mesh[Service mesh - mTLS giua workload]
        S1[Checkout + sidecar PEP]:::wl
        S2[Payment + sidecar PEP]:::wl
        S3[Order + sidecar PEP]:::wl
    end
    end
    end

    User -->|login| IdP
    User -->|request + JWT| GW
    GW -->|authz check| PDP
```



```

SPIRE -->|SVID/cert| S1 & S2 & S3
CA --> SPIRE
S1 ==>|mTLS - verify SVID| S2
S2 ==>|mTLS - verify SVID| S3
S1 -->|authz decision?| PDP
S2 -->|authz decision?| PDP

%% nguyên tắc
GW -. "deny by default, verify per-request" .- PDP

```

Hệ thống áp dụng Zero-Trust Architecture (ZTA) theo mô hình NIST SP 800-207, tách hai mặt phẳng:

- Control plane (ra quyết định): *IdP* cấp danh tính người dùng (OIDC/JWT); *SPIFFE/SPIRE* cấp danh tính workload (SVID); *PDP* (Policy Engine + Policy Administrator) đánh giá policy phân quyền; *CA* phát và xoay chứng chỉ.
- Data plane (thực thi mọi request): *PEP* tại API Gateway và sidecar của từng service enforce quyết định; mTLS bảo vệ mọi giao tiếp workload-to-workload.

Mỗi request đi qua ba kiểm tra độc lập — đừng gộp chúng: xác thực người dùng (JWT), xác thực workload (mTLS/SVID), và phân quyền (PDP). Nguyên tắc nền: *deny-by-default*, *verify per-request*, *least privilege*, *assume breach* — không tin theo vị trí mạng (ở trong VPC không đồng nghĩa được tin).

⚠ Target vs Current state: ZTA là hành trình nhiều giai đoạn. *Hiện trạng*: mTLS qua service mesh đã có; PEP tại Gateway đã có. *Đang triển khai*: PDP tập trung + per-request authz ở sidecar (giai đoạn 2); SPIRE federation (giai đoạn 3). Mỗi cột mốc ZTA là một thay đổi kiến trúc → có ADR riêng. Sơ đồ trên là kiến trúc mục tiêu.

7.1 Xác thực (Authentication)

Tách hai loại danh tính — đây là điểm ZTA bổ sung so với mô hình cũ:

- **Người dùng (user)**: OIDC + JWT (RS256); IdP (control plane) phát token; access TTL ngắn, refresh dài; MFA bắt buộc cho Admin & Merchant rút tiền; SSO Merchant tùy chọn.
- **Workload (service)**: SPIFFE/SPIRE cấp **SVID** cho mỗi workload; danh tính workload là nền tảng cho mTLS (mục 7.3) và cho authz service-to-service. Không service nào được tin chỉ vì nằm trong VPC — phải có SVID hợp lệ.

7.2 Phân quyền (Authorization)

Mô hình **PDP/PEP**: quyết định phân quyền tập trung ở **PDP** (policy-as-code), thực thi ở **PEP** (API Gateway cho request người dùng; sidecar cho gọi service-to-service), **per-request** và **deny-by-default**.

Nội dung policy là **RBAC + tenant isolation**: mọi truy vấn dữ liệu Merchant gắn `merchant_id`; Merchant A **không** truy cập dữ liệu Merchant B — kiểm tại PEP ở Gateway và tại service.

7.2.1 Role & Permission Matrix

Resource / Action	Admin	Merchant	Buyer
-------------------	-------	----------	-------

Resource / Action	Admin	Merchant	Buyer
Sản phẩm (CRUD của mình)	✓ tất cả	✓ của mình	X
Duyệt sản phẩm	✓	X	X
Đơn hàng	✓ tất cả	✓ của mình	✓ của mình
Payout/đối soát	✓	✓ xem của mình	X
Chứng từ đối soát (S3)	đọc	đọc của mình	X

7.3 Mã hóa (Encryption)

- In-transit: TLS 1.3 ở biên ngoài; mTLS giữa mọi workload nội bộ — chứng chỉ chính là SVID do SPIRE/CA cấp và xoay tự động (control plane). Đây vừa là kênh mã hóa, vừa là cơ chế xác thực workload (7.1).
- At-rest: AES-256 (KMS/Vault); STK ngân hàng & PII mã hóa field-level.
- Chứng từ đối soát: S3 + Object Lock (WORM) — quyết định: write-once, prevent overwrite/override; IAM policy chi tiết TBD (cần ADR, kèm legal-hold & retention).

7.4 Bảo mật API (PEP tại từng ranh giới)

Mỗi ranh giới tin cậy có một PEP enforce:

Ranh giới	PEP	Enforce
Public (Internet→edge)	API Gateway	Verify JWT, tenant scope, rate limit, input validation, WAF
Inter-service (VPC)	Sidecar	mTLS (verify SVID) + authz qua PDP
Outbound (PG/Bank)	Egress policy	Secret từ Vault, egress hạn chế chỉ tới đích cho phép
Inbound webhook	Gateway/handler	Verify chữ ký + allowlist IP + idempotency

Bổ sung: CORS allowlist · CSRF cho state-changing · validate server-side (không tin client).

7.5 Kiểm tra & Tuân thủ bảo mật (Fitness functions + Review)

Invariant	Cơ chế
Mọi giao tiếp service-to-service là mTLS (không plaintext)	Mesh policy / test
Không path nào bypass PEP (mọi route qua Gateway/sidecar)	Route/network policy check
SVID/chứng chỉ có xoay, không hết hạn tĩnh	Cert audit
PDP deny-by-default (không implicit allow)	Policy test
Mọi route có authn + tenant scope	Quét route/test
Webhook luôn verify chữ ký	Test
Không secret hardcode · S3 bucket bật Object Lock	Secret scan · IaC check

Invariant	Cơ chế	
Payment Svc egress chỉ tới PG/Bank	Network policy check	
Người review (gating, phán đoán)		
Loại	Ý nghĩa	Vì sao máy không kiểm soát được?
Luồng tiền mới	Thêm/sửa cách tiền đi qua hệ thống	Sai một bước có thể mất tiền thật, cần người hiểu nghiệp vụ tài chính đánh giá
Truy cập xuyên tenant	Cho phép một tenant truy cập dữ liệu/tài nguyên của tenant khác	Rủi ro rò rỉ dữ liệu cực cao, cần người đánh giá lý do hợp lệ
Thay đổi policy chứng từ bất biến	Sửa quy tắc lưu trữ chứng từ WORM	Liên quan tuân thủ pháp lý/kiểm toán, sai có thể vi phạm quy định
Mỗi cột mốc nâng cấp ZTA	Từng bước triển khai Zero Trust Architecture	Bật sai cách có thể chặn toàn bộ traffic hợp lệ hoặc ngược lại, bỏ lọt truy cập trái phép

8. HIỆU NĂNG & KHẢ NĂNG MỞ RỘNG

8.1 Yêu cầu hiệu năng

Metric	Target	Điều kiện
Checkout P99	< 800ms	Bao gồm orchestration nhiều context
Search P95	< 200ms	Catalog/ES
API P99 (khác)	< 500ms	Normal
Error rate	< 0.1%	Normal

8.2 SLA

Payment/Order 99.95% · Checkout 99.9% · Catalog/Search 99.5%.

8.3 Capacity Planning

Ước tính từ DAU × đơn/user; Payment & Order là tier-1. Headroom 3–5x.

8.4 Chiến lược Scaling

8.4.1 Horizontal

K8s HPA (CPU>70%/RPS); stateless services; read replica cho Catalog; partition Kafka theo `merchant_id`.

8.4.2 Caching

CDN (ảnh/sản phẩm) · Redis (phiên checkout, giá) · local (feature flags).

9. XỬ LÝ LỖI & KHẢ NĂNG PHỤC HỒI

9.1 Phân loại và xử lý lỗi

Chuẩn HTTP (400/401/403/404/409/422/429/500/503) — hành động chi tiết ở runbook.

9.2 Resilience Patterns

- Saga + compensation (Checkout): reserve → create order → init escrow; bước sau lỗi → release reservation + hủy pending order.
- Idempotency (bắt buộc): webhook thanh toán, payout, escrow.
- Circuit Breaker gọi cổng thanh toán/ngân hàng; Timeout mọi I/O; Graceful degradation search lỗi → cache.

9.3 Disaster Recovery (DR)

9.3.1 RTO / RPO

Tier	RTO	RPO	Gồm
1 Critical	<1h	<5min	Identity, Payment, Order
2 Business	<4h	<1h	Checkout, Inventory
3 Important	<24h	<4h	Catalog, Notification

9.3.2 Kế hoạch DR

Failover multi-AZ; restore từ backup; chứng từ S3 WORM cross-region replication; smoke test luồng tiền trước khi thông báo phục hồi; post-mortem trong 48h.

10. QUAN SÁT & GIÁM SÁT (OBSERVABILITY)

10.1 Logging

Structured JSON, mask PII/STK/secret; audit log bất biến (S3 Object Lock, 5 năm). Trường: timestamp, level, service, traceId, merchantId, requestId.

10.2 Metrics

RED per service + Golden Signals. business_order_created_total, escrow_held_total, payout_total{status}, checkout_saga_compensation_total.

10.3 Distributed Tracing

OpenTelemetry → Tempo; đặc biệt quan trọng vì checkout đi qua ≥ 4 context; 100% errors, 5% normal; propagate trace context qua gRPC + Kafka.

10.4 Alerting

Alert	Severity	Hành động
Payment fail rate >5%/5m	P1	PagerDuty
Escrow/đối soát lệch	P1	PagerDuty + freeze payout
Checkout saga compensation spike	P2	Slack + runbook
Truy cập xuyên tenant phát hiện	P1	PagerDuty + Security

10.5 Dashboard

Service Overview (SRE) · Business KPIs (PO) · SLA/SLO · **Security** (auth fail, tenant violation, audit) · **Finance** (escrow balance, payout, đối soát).

Mục 11. AI SECURITY

Không áp dụng (N/A) — hệ thống không sử dụng thành phần AI/LLM. Nếu sau này thêm (vd gợi ý sản phẩm bằng LLM), mục này bắt buộc kích hoạt theo checklist AI Security.

PHỤ LỤC

A. Tham chiếu & Tài liệu liên quan

Tài liệu	Mô tả	Link / Mã
Standard KT tổ chức	Chuẩn org-wide (log/metric/tracing)	[Link]
Tech Spec — Checkout	Thiết kế chi tiết orchestrator	TechSpec-Marketplace-Checkout.md
Tech Spec — Payment	Thiết kế chi tiết escrow/settlement	TechSpec-Marketplace-Payment.md
ADR-0012	Thêm Dispute & Refund Context	ADR-0012-them-dispute-refund-context.md
OpenAPI / proto	Hợp đồng API đầy đủ	[Link]

Bảng tương ứng SAD ↔ Tech Spec:

Mục SAD	Tech Spec chi tiết hóa
3.2.1, 4.1.1 (Checkout orchestration)	TechSpec-Marketplace-Checkout
3.2.2, 4.1.3, 6.3, 7.3 (Escrow/Settlement/WORM)	TechSpec-Marketplace-Payment

B. Bảng thuật ngữ

Escrow (giữ tiền trung gian) · Settlement (đối soát) · Payout (chi trả Merchant) · WORM (Write Once Read Many) · Saga · Bounded Context · Tenant isolation · ADR · mTLS · PII.

C. Checklist trước khi ban hành

#	Hạng mục	Đã làm
1	Sơ đồ có chú thích	☒
2	Schema/ERD review bởi DB team	☒
3	Security review (gồm tenant isolation + S3 WORM)	☒
4	SLA/SLO xác nhận bởi PO	☒
5	ADR liên quan tham chiếu (ADR-0012)	☒
9	Không có credential trong tài liệu	☒
—	Mọi mục Optional không dùng đã ghi "N/A — lý do" (Mục 11 AI)	☒

D. PHÊ DUYỆT TÀI LIỆU

Vai trò	Họ tên & Chức danh	Chữ ký / Ngày
Kiến trúc sư soạn thảo	_____	_____
Security Reviewer	_____	_____
Tech Lead / Architect	_____	_____
CTO / Head of Engineering	_____	_____